Core Language: RAII and Exceptions

RAII and scope guards, the exception-safety guarantees, `noexcept`, and choosing between exceptions and return values. Assumes Basics and OOP; move semantics, smart pointers, lambdas and types follow in their own lessons.

RAII: the central idea

RAII: tie a resource (memory, file, lock, socket) to an object. Acquire in the constructor, release in the destructor. Destructors run even when an exception is thrown, so nothing leaks.

```
class File {
    std::FILE* f_;
public:
    explicit File(const char* path) : f_(std::fopen(path, "r")) {}   // acquire
    ~File() { if (f_) std::fclose(f_); }                              // release
    File(const File&) = delete;                                       // one owner only
    File& operator=(const File&) = delete;
};
```

Watch a throw in the middle:

STEP THROUGH **RAII and stack unwinding**

```
1  void work() {
2      File f("data.txt");
3      risky();               // throws!
4  }
5
6  try { work(); }
7  catch (const std::exception& e) { }
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 6 `try { work(); }`	We call **work()** inside a try block.	
2	line 2 `File f("data.txt");`	**Acquire in the constructor.** The File object now owns an open file handle. prints: `File opened\n`	**`f (File) = file OPEN`**
3	line 3 `risky(); // throws!`	`risky()` throws an exception. Normal execution stops here: the lines after it will not run.	`f (File) = file OPEN`
4	line 4 `}`	**Stack unwinding.** On the way out of **work()** the runtime destroys every local, so **~File runs and closes the file**. No `finally`, no manual cleanup, no leak. prints: `File closed (destructor ran)\n`	~~`f (File) = file CLOSED`~~
5	line 7 `catch (const std::exception& e) { }`	The exception reaches the matching **catch**. The resource was already released along the way. This is exactly why RAII and exceptions work so well together. prints: `caught\n`	~~`f (File) = file CLOSED`~~

`vector`, `unique_ptr`, `lock_guard`, `fstream` are all RAII. Never manage a resource with a bare `new`/`delete`, `lock()`/`unlock()` or `fopen`/`fclose`: wrap it.

KEY POINT RAII binds resource lifetime to object lifetime: automatic, exception-safe cleanup. It replaces `try/finally` and removes the need for a garbage collector.

READ MORE

Programming: Principles and Practice (Stroustrup)

- 18.4 to 18.5 Resources and exceptions; resource-management pointers (RAII, unique_ptr) (PDF pages 542 to 553)

Exceptions and stack unwinding

```
double divide(double a, double b) {
    if (b == 0) throw std::invalid_argument("division by zero");
    return a / b;
}

try {
    divide(1, 0);
} catch (const std::invalid_argument& e) {        // catch by const reference
    std::cerr << e.what() << '\n';
} catch (const std::exception& e) {               // more general handler
goes AFTER specific ones
    std::cerr << "other: " << e.what() << '\n';
}
```

- **Throw by value, catch by `const&`** (catching by value slices a derived exception).
- **Stack unwinding**: leaving scopes runs destructors, in reverse order of construction, until a `catch` matches. This is why RAII is exception-safe.
- A **destructor must never throw**: if it throws while another exception is unwinding, `std::terminate` is called. Destructors are implicitly `noexcept`.
- A constructor that throws leaves no object: the destructors of already-built members and bases run, its own does not.
- `catch (...)` catches everything; rethrow with a bare `throw;` (never `throw e;`, which copies and slices).
- Derive your own exceptions from `std::exception` (or a standard subclass) so generic handlers still work.

> COST The no-throw path is almost free, but a throw is orders of magnitude slower than a return and its latency is unpredictable. Do not throw on a hot path or as ordinary control flow (see Exceptions, error codes and optional).

> READ MORE
>
> **C++ Primer (Lippman, Lajoie, Moo)**
>
> - 5.6 try blocks and exception handling (PDF pages 222 to 229)
> - 18.1 Exception handling (PDF pages 801 to 813)
>
> **Programming: Principles and Practice (Stroustrup)**
>
> - 4.5 to 4.6 Run-time errors and exceptions (PDF pages 102 to 111)
> - 18.4 to 18.5 Resources and exceptions; resource-management pointers (RAII, unique_ptr) (PDF pages 542 to 553)

RAII everywhere: guards and standard wrappers

Almost every resource has a standard RAII wrapper. Use it instead of a manual acquire/release pair.

RESOURCE	RAII TYPE	RELEASED BY

heap memory	`vector`, `string`, `unique_ptr`	destructor frees the block
a file	`std::fstream`	destructor closes it
a mutex	`lock_guard`, `unique_lock`, `scoped_lock`	destructor unlocks
a thread	`std::jthread` (C++20)	destructor joins
anything else	your own **scope guard**	destructor runs a cleanup

```
template <typename F>
class ScopeExit {  // runs F when the scope ends, however it ends
    F f_;
    bool active_ = true;
public:
    explicit ScopeExit(F f) : f_(std::move(f)) {}
    ScopeExit(const ScopeExit&) = delete;  // a guard must not be copied
    void release() { active_ = false; }  // commit: cancel the cleanup
    ~ScopeExit() { if (active_) f_(); }
};

void transfer() {
    begin();
    ScopeExit rollback([] { undo(); });  // runs on return AND on an
exception
    step1(); step2();  // any of these may throw
    rollback.release();  // success: keep the changes
}
```

STEP THROUGH **A scope guard rolls back on an exception**

```
1  void transfer() {
2      begin();
3      ScopeExit rollback(undo);
4      step1();
5      step2();                  // throws!
6      rollback.release();
7  }
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 2 `begin();`	`begin()` opens a transaction. From here on, an early exit must undo it.	**`db = transaction open`**
2	line 3 `ScopeExit rollback(undo);`	The **guard** is a local object. Its destructor will call `undo` unless it was released.	`db = transaction open` | **`rollback = active: will run undo()`**
3	line 4 `step1();`	`step1()` succeeds and changes the database.	**`db = step 1 applied`** | `rollback = active: will run undo()`
4	line 5 `step2(); // throws!`	`step2()` **throws**. The lines after it, including `release()`, never run.	`db = step 1 applied` | `rollback = active: will run undo()`
5	line 6 `rollback.release();`	**Stack unwinding** destroys `rollback`; its destructor sees it is still active and runs `undo()`. The database is back to its original state. On the success path, `release()` would have deactivated the guard first, so the changes would stay.	**`db = rolled back`** ~~| `rollback = destroyed: undo() ran`~~

- A RAII class owns **one** resource. Copying it must be **deleted** (one owner) or a **deep copy**; moving transfers ownership (the Rule of Five, OOP lesson; Move Semantics).
- Prefer the **Rule of Zero**: hold resources in members that are already RAII, and you write no destructor at all.
- **Destructors must not throw** (see Exceptions and stack unwinding and Exception-safety guarantees).

KEY POINT "What is RAII?" The resource's lifetime is tied to an object: acquire in the constructor, release in the destructor. Destructors run on every exit path, including exceptions, so nothing leaks and no `finally` is needed.

Exception-safety guarantees and noexcept

LEVEL	PROMISE IF AN EXCEPTION ESCAPES	TYPICAL EXAMPLE
No-throw	it never throws	destructors, `swap`, moves, anything `noexcept`
Strong	state is exactly as before (commit or rollback)	`vector::push_back` when the element's move is `noexcept`
Basic	nothing leaks and invariants hold, but values may have changed	most operations

Getting the strong guarantee: build aside, then swap

```
void Table::replaceAll(const std::vector<Row>& fresh) {
    std::vector<Row> copy(fresh);  // may throw: *this is still untouched
    rows_.swap(copy);  // commit: swap cannot throw
}
```

Do everything that can fail on a **copy**, then publish the result with an operation that cannot throw. If anything throws before the swap, the object is unchanged.

noexcept

- `noexcept` promises a function will not throw. If it does anyway, the program calls `std::terminate`: there is no unwinding past it.
- Destructors are `noexcept` by default. A throwing destructor during unwinding means two exceptions at once, and the program terminates.
- **Mark move operations and `swap` `noexcept` when they cannot throw.** `std::vector` grows by moving its elements *only if* the move constructor is `noexcept`; otherwise it **copies**, to keep the strong guarantee. Compiled example: growing a vector whose elements have a `noexcept` move made 0 copies; with a possibly-throwing move it copied every element.
- `noexcept(expr)` asks the compiler whether an expression can throw; generic code uses it to pick the cheap path (`std::move_if_noexcept`).
- Do not mark a function `noexcept` just to be fast: a wrong promise turns an error into a crash.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 18.1 Exception handling (PDF pages 801 to 813)

Effective Modern C++ (Meyers)

- Item 14, Declare functions noexcept if they won't emit exceptions (PDF pages 108 to 115)

C++ Move Semantics (Josuttis)

- 7.1 and 7.4 Move constructors with and without noexcept; when and where to use noexcept (PDF pages 125 to 134 and 140 to 141)

Exceptions, error codes and optional

SITUATION	USE

a failure the caller cannot sensibly handle here (out of memory, broken invariant, constructor failure)	**exception**: it unwinds to code that can cope
an **expected** failure in normal flow (bad input, "not found", parse error)	return a value that says so: `std::optional`, an error code or enum, `std::expected` (C++23)
a hot path with steady latency (real-time code)	**no exceptions**: error codes or `optional` (Performance & Memory)
a `noexcept` context, or a destructor	never throw: catch inside and log or ignore

```
std::optional<int> parseInt(std::string_view s) {  // failure is expected: no exception
    int v = 0;
    auto r = std::from_chars(s.data(), s.data() + s.size(), v);
    if (r.ec ≠ std::errc{} || r.ptr ≠ s.data() + s.size()) return std::nullopt;
    return v;
}

if (auto n = parseInt(input)) use(*n);  // the caller must look at the result
else report("not a number");
```

- **Exceptions** keep the normal path clean and cannot be silently ignored; the cost is that a throw is slow and the control flow is invisible.
- **Return values** make failure explicit and cheap; the cost is that every caller must check, and `[[nodiscard]]` helps remind them.
- Constructors cannot return a code: either throw, or use a factory function returning `optional` or an `expected`.
- Be consistent inside a module, and document what a function can throw.

KEY POINT "Exceptions or error codes?" Exceptions for rare failures that need unwinding, return values for expected failures and for hot paths. Mention RAII (so nothing leaks either way) and that a destructor must never throw.

Cheat sheet

Answer aloud, then open.

<details>
<summary>What is RAII and why is it important?</summary>

It binds a resource's lifetime to an object: acquire in the constructor, release in the destructor. Cleanup is automatic and exception-safe because destructors run on every exit path. It replaces `try/finally` and is why C++ needs no garbage collector.
</details>

<details>
<summary>What is stack unwinding?</summary>

When an exception propagates, the runtime exits scopes up the call stack, calling the destructors of all fully constructed locals until a handler is found. It is what makes RAII exception-safe.
</details>

<details>
<summary>Why should destructors not throw?</summary>

Destructors run during unwinding. If one throws while another exception is active, `std::terminate` is called. Destructors are `noexcept` by default; catch and handle errors inside them.
</details>

<details>
<summary>What are the exception-safety guarantees?</summary>

No-throw (never throws: destructors, swap, moves), strong (if it throws, the state is exactly as before: commit or rollback), and basic (nothing leaks and invariants hold, but values may have changed). Get the strong guarantee by doing the work on a copy and publishing with a non-throwing swap.
</details>

▼ Why mark move operations `noexcept`?

`std::vector` moves its elements when it grows only if the move cannot throw; otherwise it copies them to keep the strong guarantee. A missing `noexcept` silently turns every reallocation into a round of copies.

▼ What happens if a `noexcept` function throws?

`std::terminate` is called: the exception does not propagate. So only promise `noexcept` when it is true.

▼ Exceptions or error codes?

Exceptions for rare failures the local code cannot handle (and constructors); return values (`optional`, error codes, `expected`) for expected failures and for hot paths where a throw's cost and unpredictability are unacceptable.

▼ How do you run cleanup on every path without `finally`?

Put it in a destructor: a standard RAII wrapper (`lock_guard`, `unique_ptr`, `fstream`) or a small scope guard class. Release the guard on success to commit.